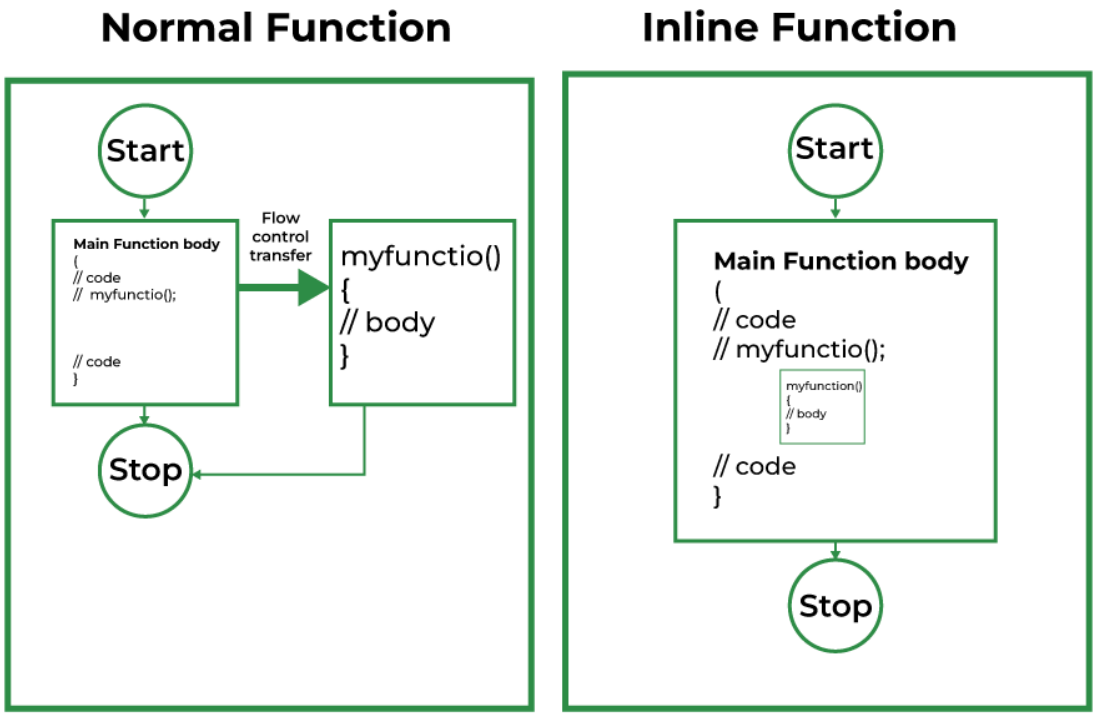
1. Inline Function

Definition:

An **inline function** is a function whose definition is **expanded at the point of invocation** instead of performing a function call. The `inline` keyword suggests the compiler to replace the function call with the actual function code, reducing overhead.



Use Case:

- Improves performance for **small, frequently used functions**.
- Eliminates the overhead of function calls (stack push/pop operations).
- Useful for simple functions like **getters, setters, and mathematical operations**.

Syntax:

```
inline return_type function_name(parameters) {
    // Function body
```

```
}
```

Example 1 (Basic Level) – Inline Function for Multiplication

```
#include <iostream>
using namespace std;

// Inline function
inline int multiply(int a, int b) {
    return a * b;
}

int main() {
    cout << "Multiplication of 5 and 3: " << multiply(5, 3) << endl;
    return 0;
}
```

Output:

Multiplication of 5 and 3: 15

Example 2 (Medium Level) – Inline Function for Calculating Area

```
#include <iostream>
using namespace std;

class Rectangle {
    int length, width;

public:
    // Parameterized constructor
    Rectangle(int l, int w) : length(l), width(w) {}

    // Inline function to calculate area
    inline int getArea() {
        return length * width;
    }

    void display() {
        cout << "Area of rectangle: " << getArea() << endl;
    }
};

int main() {
    Rectangle rect(10, 5);
    rect.display();
    return 0;
```

```
}
```

Output:
Area of rectangle: 50

---

2. Static Data Members & Static Member Functions

Definition:

- **Static Data Members**: A **class-level variable** that is shared across all objects. Only one copy exists, regardless of the number of objects created.
- **Static Member Functions**: A function that can **access only static data members** and can be called without creating an object.

Use Case:

- Used for **tracking the count** of objects created.
- Shared values among **all instances** of a class.
- **Utility functions** that do not depend on instance variables.

Syntax:

```
class ClassName {
    static int staticVar;   // Declaration of static variable

public:
    static void staticFunction() {
        // Can access only static members
    }
};
```

Example 1 (Basic Level) – Counting Objects Using Static Data Members

```
#include <iostream>
using namespace std;

class Counter {
    static int count;  // Static data member

public:
    Counter() {
        count++;  // Increment count whenever an object is created
    }
```

```
    static void displayCount() {  // Static function
        cout << "Number of objects created: " << count << endl;
    }
};

// Initialize static data member
int Counter::count = 0;

int main() {
    Counter obj1, obj2, obj3;
    Counter::displayCount();  // Call static function
    return 0;
}
```

Output:

Number of objects created: 3

Example 2 (Medium Level) – Static Function to Set and Get Data

```
#include <iostream>
using namespace std;

class Bank {
    static int totalCustomers;

public:
    static void addCustomer() {
        totalCustomers++;
    }

    static void showTotalCustomers() {
        cout << "Total bank customers: " << totalCustomers << endl;
    }
};

// Initialize static member
int Bank::totalCustomers = 0;

int main() {
    Bank::addCustomer();
    Bank::addCustomer();
    Bank::addCustomer();

    Bank::showTotalCustomers();
    return 0;
}
```

Output:
Total bank customers: 3

---

3. Objects as Function Arguments

Definition:

Objects can be passed **by value** or **by reference** to a function, allowing operations on the object's data.

Use Case:

- Used for **copying objects**, performing **comparisons**, or **manipulating object properties**.
- Allows **function interactions** with **class objects**.

Syntax:

```
class ClassName {
public:
    void functionName(ClassName obj) {
        // Use object inside function
    }
};
```

Example 1 (Basic Level) – Passing Object to Function

```
#include <iostream>
using namespace std;

class Car {
public:
    string brand;

    Car(string b) {
        brand = b;
    }

    void display(Car c) {
        cout << "Car brand: " << c.brand << endl;
    }
};

int main() {
    Car car1("Toyota");
```

```
    car1.display(car1);
    return 0;
}
```

Output:
Car brand: Toyota

Example 2 (Medium Level) – Comparing Two Objects

```
#include <iostream>
using namespace std;

class Student {
    int marks;

public:
    Student(int m) {
        marks = m;
    }

    void compare(Student s) {
        if (marks > s.marks)
            cout << "Current student has higher marks" << endl;
        else
            cout << "Passed student has higher or equal marks" << endl;
    }
};

int main() {
    Student s1(85), s2(90);
    s1.compare(s2);
    return 0;
}
```

Output:
Passed student has higher or equal marks

---

4. Friend Functions

Definition:

A **friend function** is **not a member of a class** but has **access to private and protected members** of that class. It is declared using the `friend` keyword inside the class.

Friend Function in C++



Use Case:

- Allows **external functions** to access **private data**.
- Helps when **two classes need to share data** without inheritance.

Syntax:

```
class ClassName {
   friend return_type functionName(ClassName obj);
};
```

Example 1 (Basic Level) – Friend Function Accessing Private Members

```
#include <iostream>
using namespace std;

class Box {
private:
   int width;

public:
   Box(int w) {
      width = w;
   }

   // Friend function declaration
   friend void displayWidth(Box b);
};
```

```
// Friend function definition
void displayWidth(Box b) {
    cout << "Width: " << b.width << endl;
}

int main() {
    Box b1(10);
    displayWidth(b1);
    return 0;
}
```

Output:

Width: 10

Example 2 (Medium Level) – Adding Two Objects Using Friend Function

```
#include <iostream>
using namespace std;

class Account {
private:
    int balance;

public:
    Account(int b) {
        balance = b;
    }

    friend int addBalance(Account a1, Account a2);
};

// Friend function definition
int addBalance(Account a1, Account a2) {
    return a1.balance + a2.balance;
}

int main() {
    Account acc1(5000), acc2(7000);
    cout << "Total Balance: " << addBalance(acc1, acc2) << endl;
    return 0;
}
```

Output:

Total Balance: 12000
